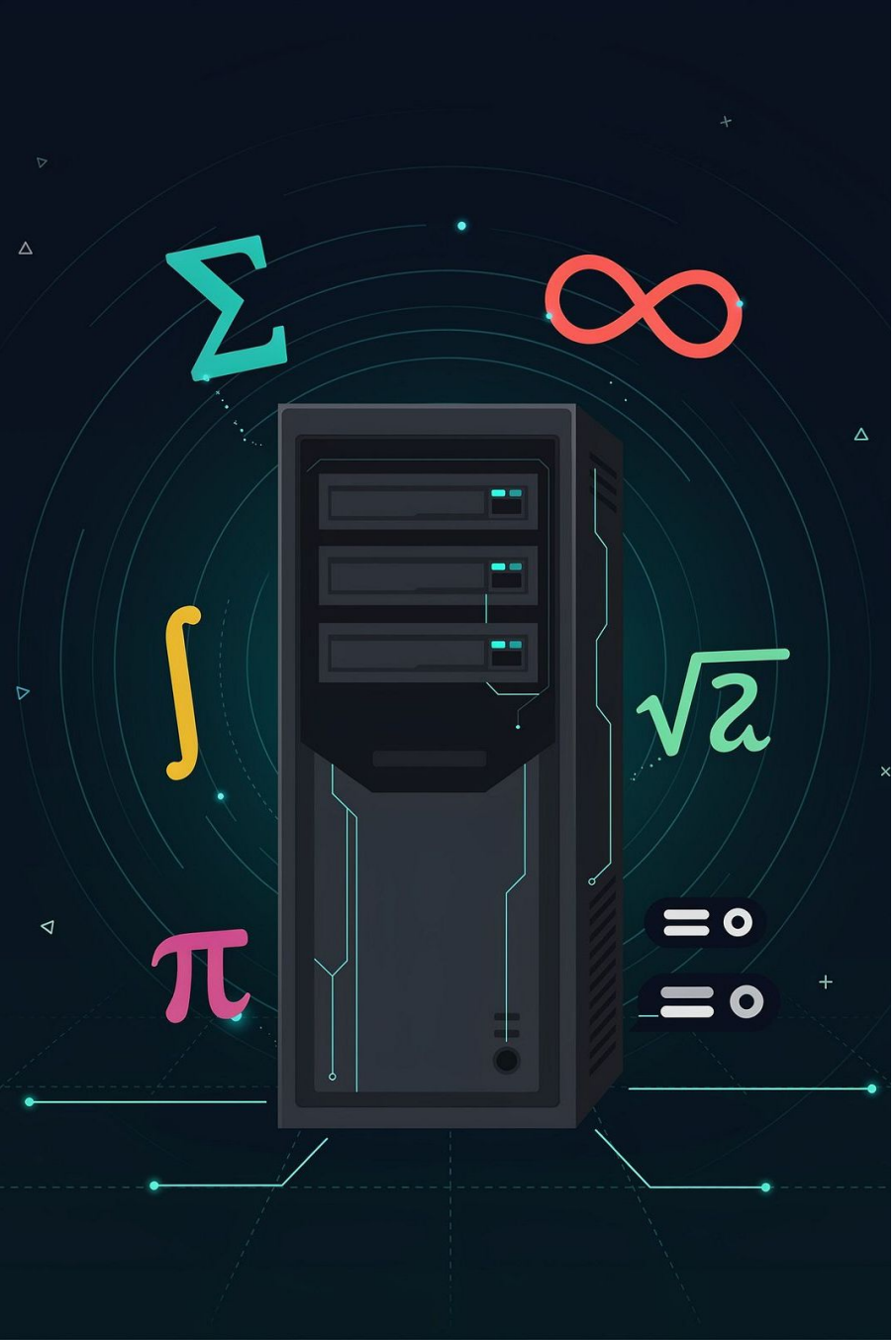


SQLite: Cálculos e Agregações na Prática

SQLITE · ESTOQUE · SETORES · RELATÓRIOS

Operadores Aritméticos Básicos

Operações aritméticas servem para efetuar cálculos, como valor em estoque, lucro por unidade e preço promocional diretamente na query. Isso elimina a necessidade de calcular esses valores na aplicação, tornando o código mais limpo e eficiente.

+ Soma

Soma entradas de estoque de um mesmo produto

– Subtração

Lucro unitário: preço – custo

× Multiplicação

Valor em estoque: preço × estoque

÷ Divisão

Preço por unidade em fardos ou pacotes — atenção ao tipo de dado!

```
SELECT
  nome,
  setor,
  preco,
  estoque,
  preco * estoque AS valor_em_estoque,
  preco - custo AS lucro_unitario,
  preco * (1 - desconto / 100.0) AS preco_promocional
FROM produto;
```

⚠ **Dica importante:** Use `100.0` em vez de `100` na divisão. No SQLite, a divisão entre dois inteiros retorna um inteiro, truncando a parte decimal. Por isso, um desconto de 15% cadastrado como `desconto = 15` só funciona corretamente com `100.0`, gerando um preço promocional de forma precisa.

Funções de Agregação

Funções de agregação resumem dados em valores únicos, como estoque total, faturamento e preço médio. Assim, o gerente consegue acompanhar o negócio sem precisar olhar item por item.



SUM()

Soma o total de itens em estoque ou o faturamento das vendas do dia, da semana ou do mês.



AVG()

Calcula o preço médio dos produtos ou o ticket médio das compras dos clientes.



COUNT()

Conta os produtos cadastrados. Use `COUNT(*)` para contar todas as linhas, ou `COUNT(data_validade)` para ignorar produtos sem validade (NULL).



MAX() / MIN()

Retornam o produto mais caro e o mais barato, ou a validade mais próxima do vencimento.

Agregação por Setor com GROUP BY

GROUP BY separa os produtos por setor e aplica as agregações em cada grupo, mostrando o estoque do mercado de forma organizada.

Estoque por Setor

```sql
SELECT
  setor,
  COUNT(*) AS qtd_produtos,
  SUM(estoque) AS itens_em_estoque,
  ROUND(AVG(preco), 2) AS preco_medio
FROM produto
WHERE status = 'ativo'
GROUP BY setor
HAVING SUM(estoque) < 1500
ORDER BY itens_em_estoque DESC;
```

Resultado de Exemplo

setor	qtd_produtos	itens_em_estoque	preco_medio
Hortifruti	18	1.365	6.45
Limpeza	17	1.345	9.67
Laticínios	17	1.235	9.30

Dica: `WHERE` filtra linhas antes do agrupamento; `HAVING` filtra os grupos depois da agregação. Use `HAVING SUM(estoque) < 1500` para achar setores com estoque abaixo de 1.500 itens.

Funções Matemáticas

Funções matemáticas ajudam a precificar, reajustar e organizar pedidos de reposição no dia a dia do mercadinho.

POW(x, y)

Reajuste composto de preço. Ex.: aumento de **5% ao mês** por 3 meses.
`POW(1.05, 3)` → **1.157625** (alta de 5% ao mês por 3 meses)

CEIL(x)

Caixas necessárias na reposição, sempre arredondando **para cima**.
`CEIL(25 / 12.0)` → **3 caixas** para 25 unidades, com 12 por caixa

FLOOR(x)

Pacotes ou combos completos que podem ser montados, arredondando **para baixo**.
`FLOOR(25 / 12.0)` → **2 combos** completos

ROUND(x, d)

Arredonda o preço para casas decimais.
`ROUND(10.456, 2)` → **10.46** (preço final na etiqueta)

Comparando Funções de Arredondamento

Veja como cada função transforma o mesmo preço **R$ 10,45** (quilo do tomate) de forma diferente. Escolher a função de arredondamento certa evita erros em preços e etiquetas.

```
SELECT
  nome,
  preco / 100.0,
  CEIL(preco / 100.0),
  FLOOR(preco / 100.0),
  ROUND(preco / 100.0, 1),
  ROUND(preco / 100.0, 0),
  ROUND((preco / 100.0) * POW(1.05, 2), 2)
FROM produto
WHERE nome = 'Tomate';
```

Função	Entrada	Saída
CEIL()	10.45	**11**
FLOOR()	10.45	**10**
ROUND(x, 1)	10.45	**10.4**
ROUND(x, 0)	10.45	**10**
ROUND(x * POW(1.05, 2), 2)	10.45	**11.52**

Sempre teste essas funções com os valores reais antes de gerar etiquetas ou relatórios. Diferenças de arredondamento podem parecer pequenas, mas se multiplicam em grandes volumes de dados.

Combinando Tudo: Views para Relatórios do Mercado

A verdadeira força do SQLite emerge quando combinamos operadores aritméticos, funções de agregação e funções matemáticas dentro de **Views**. No mercadinho, uma View pode gerar o relatório de estoque por setor, centralizando as regras de cálculo no banco de dados.

Criando uma View de Estoque por Setor

```
CREATE VIEW vw_relatorio_estoque_setor AS
SELECT
  setor,
  COUNT(*) AS qtd_produtos,
  SUM(estoque) AS itens_em_estoque,
  ROUND(SUM(preco * estoque), 2) AS valor_em_estoque
  FROM produto
GROUP BY setor;
-- Consulta simplificada:
SELECT * FROM vw_relatorio_estoque_setor
ORDER BY valor_em_estoque DESC;
```

Por que usar Views?

Reutilização de lógica

O gerente consulta o relatório de estoque sem precisar reescrever a query toda vez.

Simplicidade nas consultas

O painel de estoque da loja acessa os valores calculados sem repetir fórmulas.

Manutenção centralizada

Se a regra de cálculo do valor em estoque mudar, basta alterar a View em um único lugar.

Resumo: O que Você Aprendeu

Neste mercadinho, cobrimos os recursos de cálculo do SQLite aplicados ao dia a dia do negócio — de operadores simples a Views de relatório para o gerente.

Operadores Aritméticos

Calcule valor em estoque, lucro unitário e preço promocional. Use `.0` para evitar truncamento em divisões.

Funções de Agregação e GROUP BY



SUM, AVG, COUNT, MAX e MIN para resumir estoque e vendas por setor.

Funções Matemáticas

POW, CEIL, FLOOR e ROUND para reajustes de preços e cálculo de caixas de reposição.

Views Inteligentes



Relatórios de estoque por setor prontos para o gerente consultar a qualquer momento.

Com esses recursos você consegue controlar estoque, setores e preços do seu mercadinho direto no SQLite.